

SOFTWARE ENGINEERING LABORATORY

GitStack Technical Report

A Git Learning and Team Collaboration Platform

Software Engineering Laboratory | 9th Trimester | 2026

Project Team

Jubair Ahmed Efty | Shabab Ahmed | Siam Ibne Sarwar

United International University

4 October 2026

Table of Contents

Abstract.....	3
1. Project Overview.....	4
2. System Overview and Architecture.....	5
3. Feature-wise Technical Implementation.....	7
4. Database and Data Management.....	22
5. API and Backend Design.....	24
6. Frontend and Client-Side Implementation.....	25
7. Security and Error Handling.....	26
8. Testing and Validation.....	26
9. Deployment and Setup.....	27
10. Team Contribution.....	29
11. Challenges and Technical Decisions.....	29
12. Conclusion and Future Improvements.....	30
13. Project Links and References.....	30

Abstract

GitStack is a web-based Git learning platform designed for software engineering laboratory use. It connects short lessons with individual missions executed inside isolated Docker workspaces. Students complete ordered Git tasks, receive state-aware hints, earn attempt-based XP, and submit repository state for automatic assessment. Instructors can author and publish supported missions, assign work to students or three-person teams, and review progress, assessment evidence, and feedback. Team assignments connect to a real Gitea organization repository: each member receives a role and branch, while pushes, pull requests, reviews, test evidence, and merge order contribute to the workflow report. The platform uses HTML, CSS, vanilla JavaScript, Node.js and Express, PostgreSQL with Prisma, Docker, and Gitea. English/Bangla controls and light/dark themes support the learning interface. This report documents the system scope, architecture, database, APIs, security, testing, local setup, and team responsibilities. The fresh Ubuntu setup completed locally; source, UI, sandbox, database migration, and Gitea integration checks recorded in the setup session passed. GitStack is currently a local laboratory application rather than a public cloud service.

Keywords: Git education, repository-state assessment, Docker sandbox, Gitea collaboration, dynamic hints, XP

The interface figures are selected from the 53 project still images supplied with the report materials. Values shown in sample dashboards are illustrative UI data and are not presented as measured learning outcomes.

1. Project Overview

1.1 Project Idea

GitStack teaches Git through guided repository work rather than command memorization alone. A student chooses an individual mission, works in a temporary terminal-backed repository, and sees each step unlock when the required Git state is reached. Instructors can create additional missions and use a shared team workflow to teach the same branch, review, testing, and merge practices used in collaborative software development.

1.2 Problem Statement

Students often learn Git commands as isolated syntax, while real development requires understanding how commands change files, the staging area, branches, commits, and remotes. Practising directly on a personal or team repository can risk unwanted changes. Instructors also need a practical way to observe progress and assess repository work consistently. GitStack addresses these needs with disposable individual workspaces, state-based validation, guided hints, and a real Gitea-backed team exercise.

1.3 Objectives

- Provide separate student and instructor accounts with role-specific workspaces.
- Teach Git through ordered missions that validate observable repository state.
- Let instructors author, publish, and assign supported individual missions without editing source code.
- Offer step-aware Clue, Guidance, and Answer hints with transparent XP costs.
- Run student terminal work in isolated Docker containers and preserve mission history in PostgreSQL.
- Support a real three-person Gitea workflow with branches, issues, pull requests, reviews, tests, and merges.
- Give instructors assessment, feedback, progress, and activity views, and give students optional visual and vocabulary practice.

1.4 Target Users

Students use the lesson pages, individual mission workspaces, progress and XP views, assessment feedback, team activity, and supplementary learning tools. Instructors create and assign missions, manage teams, configure team repositories, and review individual or team evidence. A local platform administrator configures the application, databases, Docker services, and Gitea integration.

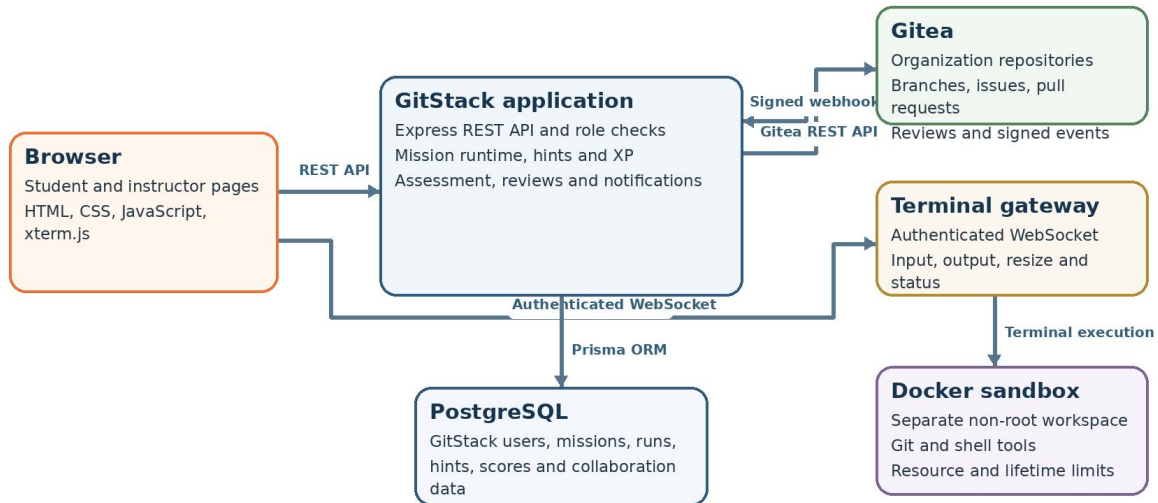
1.5 Scope

The delivered scope includes role-based web pages, individual mission authoring and execution, repository-state assessment, hints and XP, Docker sandbox management, team assignments with Gitea, instructor analytics, feedback and work reviews, Git Flow Lab, and Git Hangman. The current deployment target is a Docker-capable laboratory host. The project does not claim a public cloud deployment, semantic code-quality grading, a general-purpose CI service, or measured improvement in student learning outcomes.

2. System Overview and Architecture

2.1 System Architecture

GitStack System Architecture



Individual sandboxes have no network access; collaboration sandboxes use the restricted network needed for Gitea.

Figure 1. Browser, application, database, Gitea, WebSocket terminal, and Docker sandbox components.

The browser communicates with the Express application through role-protected JSON APIs. The terminal client uses an authenticated WebSocket to send input and receive output from an owned sandbox. The application persists users, missions, attempts, hint purchases, assessment results, and team metadata in PostgreSQL through Prisma. It calls Gitea for repository and collaboration operations; Gitea sends signed webhooks back to GitStack. The application validates mission progress and workflow assessment on the server so the browser does not decide scores or permissions. Individual mission sandboxes have no network access; team sandboxes use a restricted network path to Gitea. [1, 2, 3]

2.2 Technology Stack

Table 1. Main implementation technologies.

Layer	Technology	Role in GitStack
Frontend	HTML5, CSS3, vanilla JavaScript	Role dashboards, learning pages, forms, mission controls, and responsive UI.
Browser terminal	xterm.js and authenticated WebSocket	Displays terminal output and connects students to their assigned sandbox.
Backend	Node.js 20+ and Express 5	REST APIs, authentication, mission lifecycle, validation, assessment, and integrations.
Input validation	Zod and repository-state checks	Validates API payloads and checks real Git state against mission rules.
Database	PostgreSQL 17 and Prisma 6	Stores users, missions, runs, teams, XP, assessments, reviews, and notifications.

Layer	Technology	Role in GitStack
Sandbox	Docker with a Debian-based image	Creates resource-limited terminal workspaces for individual and team exercises.
Collaboration	Gitea 1.24 REST API and signed webhooks	Hosts real organization repositories, issues, branches, pull requests, and review evidence.
Security	Argon2id, JWT cookies, AES-256-GCM, HMAC	Protects passwords, sessions, selected profile fields, and webhook events.

2.3 Development Tools and Services

Table 2. Development and runtime tools.

Tool or service	Purpose
Git and GitHub	Version control and source collaboration; the project source is hosted in the team repository.
Docker Engine and Compose	Runs PostgreSQL, Gitea, and disposable student sandboxes.
Prisma CLI and migrations	Validates the schema, generates the client, and applies database changes.
npm scripts	Installs pinned dependencies and runs setup, regression checks, doctors, and the development server.
Gitea	Provides the real Git hosting service used by collaboration missions.

2.4 Repository and Deployment

The source repository is hosted on GitHub. The tested installation described in this report runs on a local Ubuntu machine: Node.js serves GitStack on port 3000, Docker Compose runs PostgreSQL and Gitea, and student workspaces are created as Docker containers. No public application URL is configured.

Table 3. Project links and local services.

Item	Address or deployment note
GitHub repository	https://github.com/Shabab122/GitStack
GitStack local application	http://localhost:3000
Gitea local service	http://localhost:3002
Deployment platform	Local Ubuntu laboratory host with Docker Compose; no public deployment.

3. Feature-wise Technical Implementation

The feature set is implemented as connected student, instructor, mission, sandbox, assessment, collaboration, and supplementary learning modules. The following sections describe the user flow and the backend behavior that supports each visible page.

3.1 Authentication and Role-Based Dashboards

Feature Overview

GitStack provides student and instructor registration and login, then opens a workspace appropriate to the selected role. Students see their learning activity and quick links; instructors see class progress, missions, assignments, teams, assessments, and analytics.

User Flow and Workflow

A user registers with a role and account details, logs in, and receives a signed session cookie. The student dashboard links to missions, progress, feedback, team activity, and profile pages. The instructor dashboard links to student records, mission management, assignments, teams, Gitea, collaboration reports, assessments, analytics, and activity.

Technologies Used

HTML, CSS, vanilla JavaScript, Express, JWT, Argon2id, PostgreSQL, and Prisma.

Technical Implementation

Authentication endpoints establish the session and role. Protected routes check the authenticated user and role before reading or changing records. Shared frontend scripts provide language and theme controls, while separate student and instructor pages keep each workflow understandable.

Feature Screenshot and Output

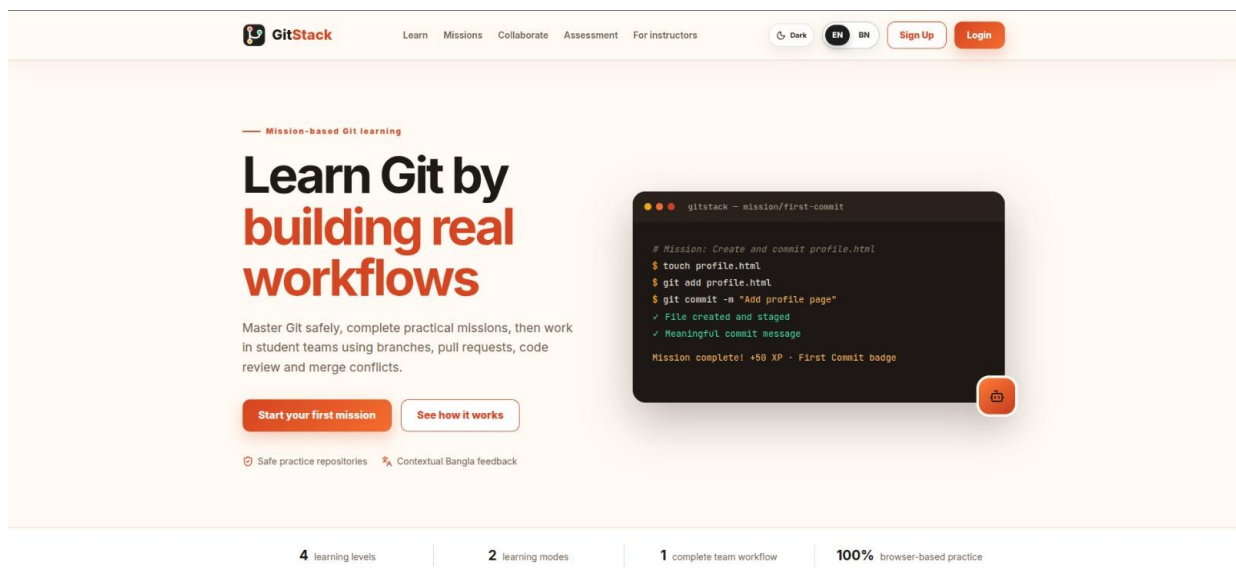


Figure 2. Public landing page introducing GitStack and the learning workflow.

JOIN GITSTACK

Create your learning workspace.

Choose the correct role, create your account and continue into the GitStack mission environment.

- Secure password handling
- Isolated student workspaces

STUDENT ACCOUNT

Create your account

Use accurate information. Your role controls which dashboard and workflow tools you can access.

Full name
 University ID

University email

Department
 Semester / designation

Create a strong password

Use at least 8 characters with uppercase, lowercase, a number and a symbol.

Confirm password

Figure 3. Student registration screen and role entry point.

WELCOME BACK

Continue your GitStack workflow.

Sign in with your selected role to access missions, progress, collaboration activity or instructor tools.

- Continue guided missions
- Review collaboration activity
- View progress and assessment

STUDENT ACCOUNT

Login to GitStack

Enter your university account details to continue.

PREVIOUSLY USED ACCOUNT

Shabab
shabab@gmail.com

Your browser password manager can fill the password securely. GitStack does not store your password on this device.

Use this account

Forget

University email or ID

Password

☐ Keep me signed in on this device.

Login securely

Need an account? [Sign Up](#)

Login verification runs on the GitStack backend. Passwords are Argon2id-hashed and the authenticated session is stored in an

Figure 4. Login screen for returning students and instructors.

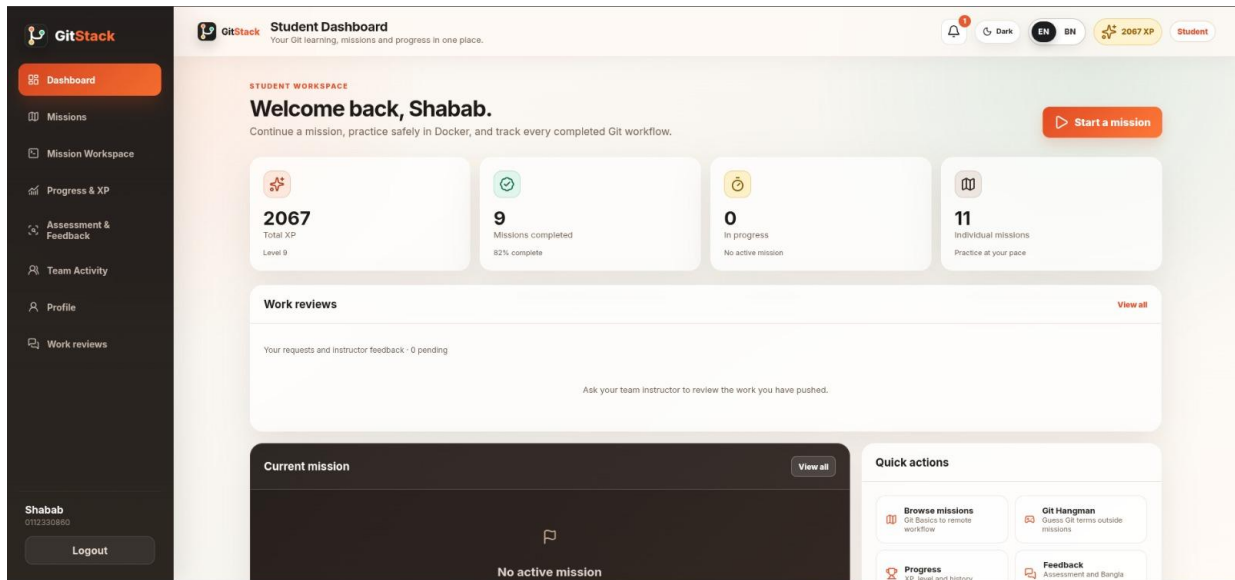


Figure 5. Student dashboard summary and quick actions. Values are sample interface data.

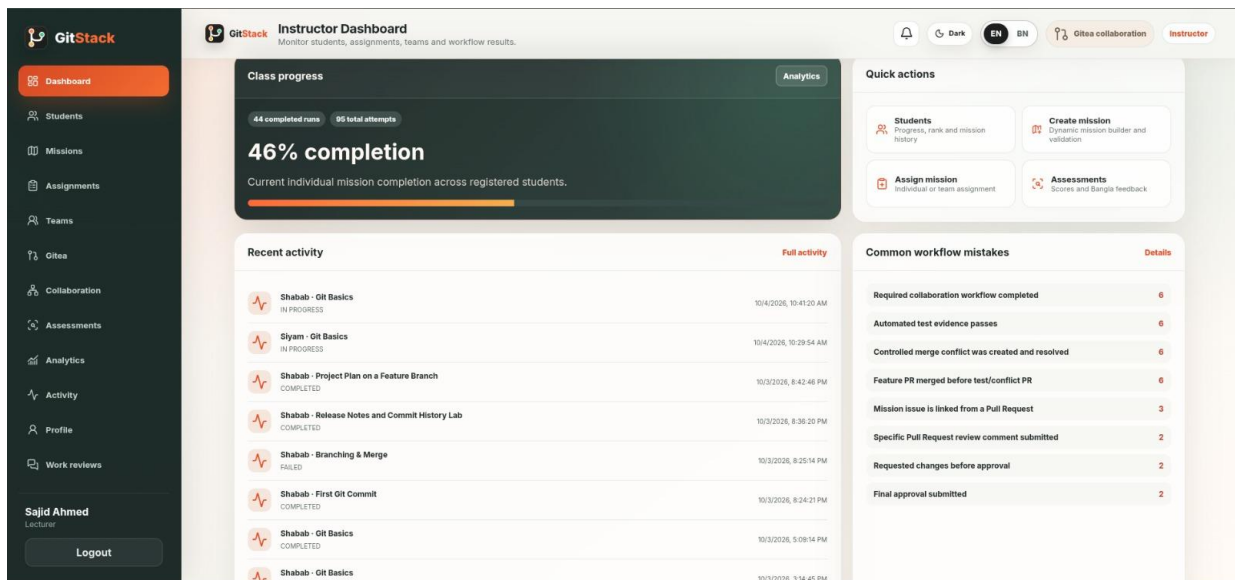


Figure 6. Instructor dashboard overview and class monitoring panels. Values are sample interface data.

Technical Notes and Limitations

GitStack credentials and Gitea credentials are separate. A student links the exact Gitea username in their profile before an instructor can synchronize repository access.

3.2 Instructor Mission Authoring and Assignments

Feature Overview

Instructors can create individual missions with an objective, ordered steps, an estimated duration, an XP reward, and automatic repository validation rules. A mission can be published for students and then assigned to one student or an eligible three-person team.

User Flow and Workflow

The instructor opens the mission catalogue, creates a mission, enters one observable action per step, selects validation rules, and publishes it. The instructor then creates an assignment, chooses an individual student or team, and sets its availability and due dates. Assignment status can be managed without changing the mission definition.

Technologies Used

Instructor dashboard pages, Express instructor APIs, Prisma mission and assignment models, Zod payload validation, and repository-state validation services.

Technical Implementation

Mission definitions are stored as MissionTemplate records. Published custom missions must satisfy the supported mission contract before students can run them. The runtime prepares a repository for the first step, then checks ordered progress. The form supports 1 to 12 steps, optional Clues, XP, level, and an estimated duration. Validation can require repository initialization, a file, tracking, commits, branch naming, final branch, or a clean working tree. [1, 4, 5]

Feature Screenshot and Output

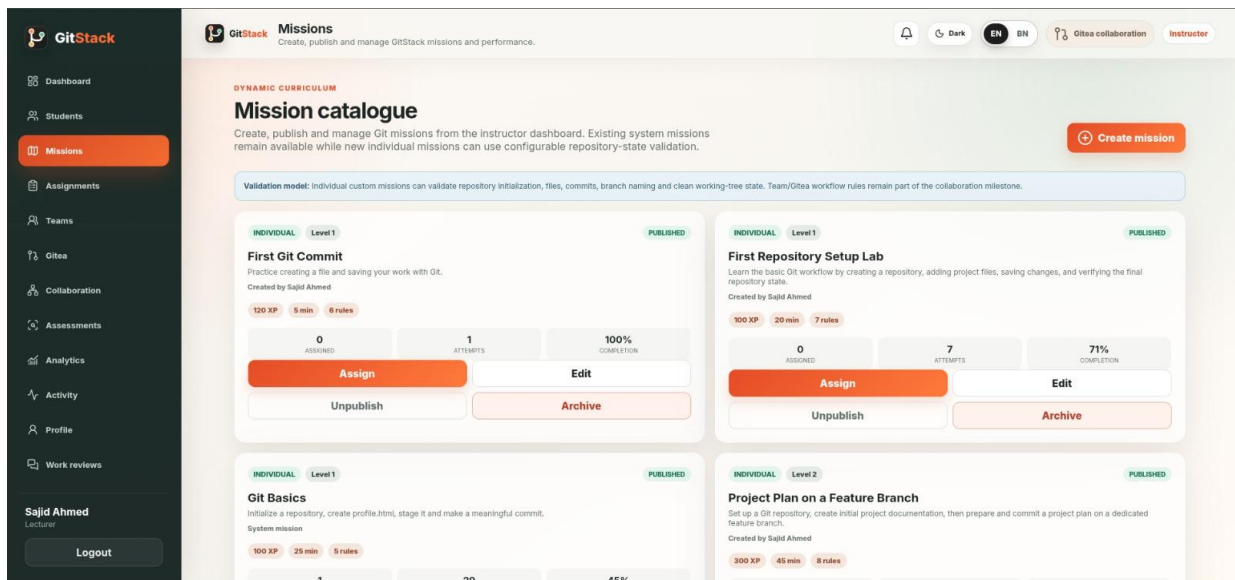


Figure 7. Instructor mission catalogue with create, assign, edit, publish, and archive controls.

Create mission
Build a mission without editing source code

Mission title
Example: Commit Quality Lab

Slug (optional)
commit-quality-lab

Description
What should the student learn and deliver?

Mission type
Individual

Level
1

XP reward
100

Estimated minutes
30

Objective
Describe the outcome, not the exact commands.

Mission steps — one objective step per line
Initialize a repository
Create the required file
Commit the completed work

Student-facing command suggestions are intentionally not shown.

Customize Clues (optional) 0 custom Clues

Figure 8. Mission creation form for title, objective, steps, reward, and estimate.

Mission steps — one objective step per line
Initialize a repository
Create the required file
Commit the completed work

Student-facing command suggestions are intentionally not shown.

Customize Clues (optional) 0 custom Clues

Automatic validation rules

☒ **Repository must be initialized**

Required file
example.md

☐ **Required file must be tracked**

Minimum commits
1

Minimum latest commit message length
8

Required branch prefix
feature/

Finish on branch
main

☐ **Working tree must be clean**

☐ **Publish mission**
Published missions become visible to students and available for assignments.

Save mission **Cancel**

Figure 9. Automatic validation rules and the publish option in the mission form.

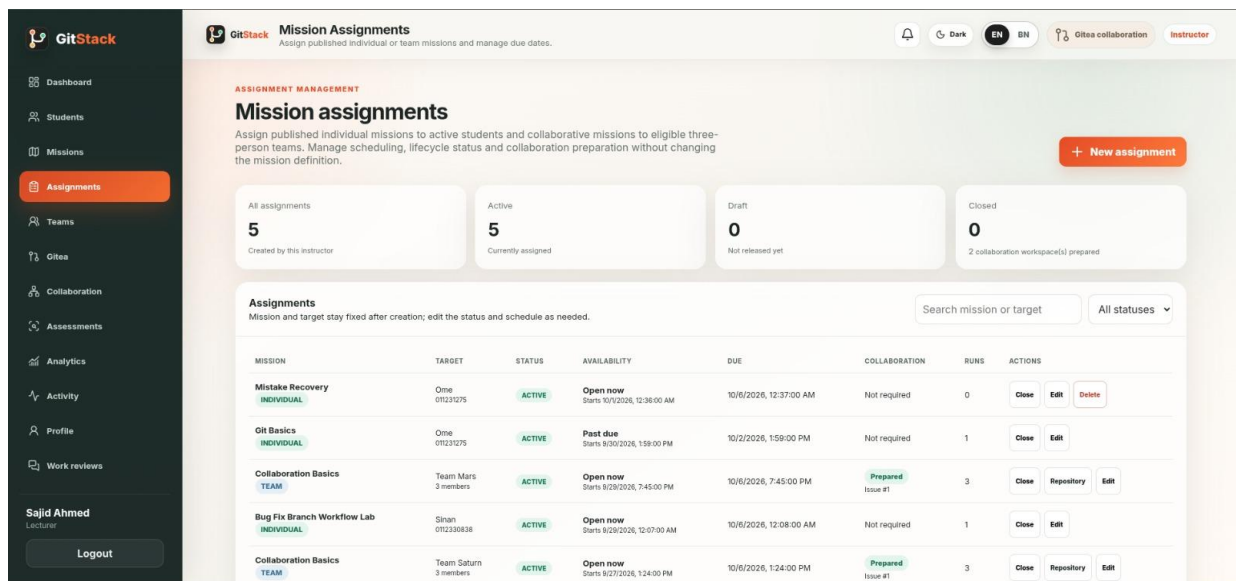


Figure 10. Assignment management list for individual and team work.

Technical Notes and Limitations

Published missions are reusable templates; assignments attach a template to a student or team. The system protects mission history by archiving custom missions that already have runs instead of erasing assessment records.

3.3 Individual Mission Workspace, Hints, and XP

Feature Overview

Students work through an individual mission in order. The mission panel shows the current step, progress, remaining time, and available hints; the terminal executes real Git commands in the associated sandbox. Completion is checked against repository state.

User Flow and Workflow

The student opens a published mission and starts an attempt. The server prepares the workspace, unlocks the first step, and receives terminal activity. A valid state completes the active step and unlocks the next one. The student can request a Clue, Guidance, or Answer for the current unfinished step, then submit the repository for final assessment.

Technologies Used

MissionRun and MissionHintUse models, repository-state inspection, a sequential step engine, Docker terminal service, authenticated WebSocket, and the dynamic hint-XP service.

Technical Implementation

The runtime checks Git state rather than requiring one exact command when multiple commands can produce the same valid result. The server stores progress and hint purchases. Final submission validates the required repository state, records AssessmentResult and Feedback, and awards only the unused reward from an eligible completion. Previously earned account XP is not removed by hint use. [1, 6]

Feature Screenshot and Output

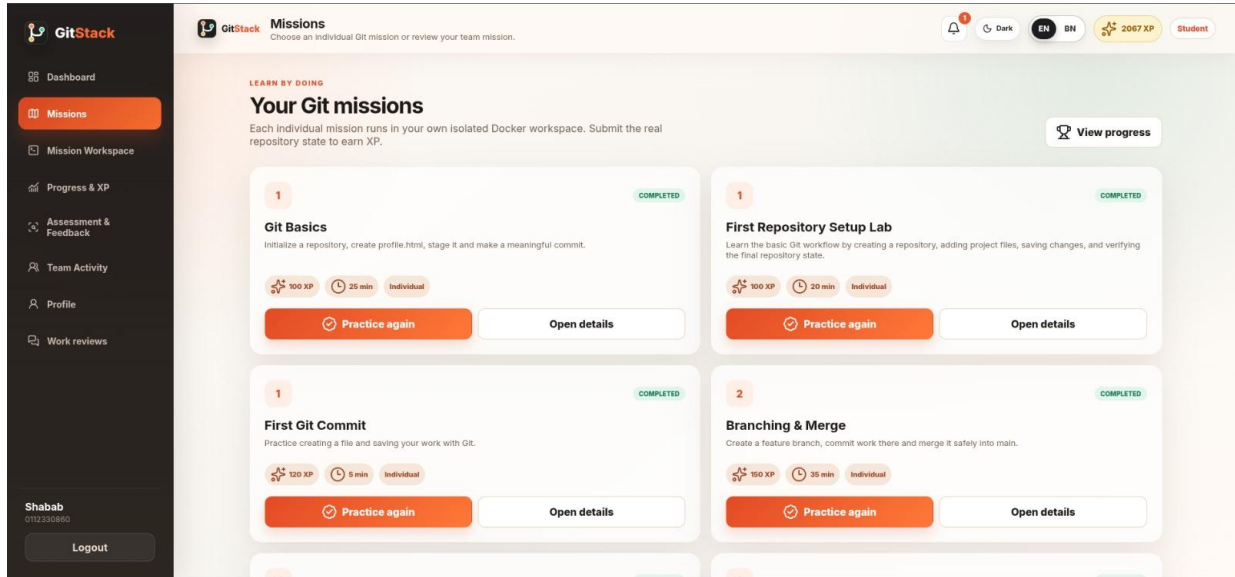


Figure 11. Student mission catalogue showing mission status and practice actions.

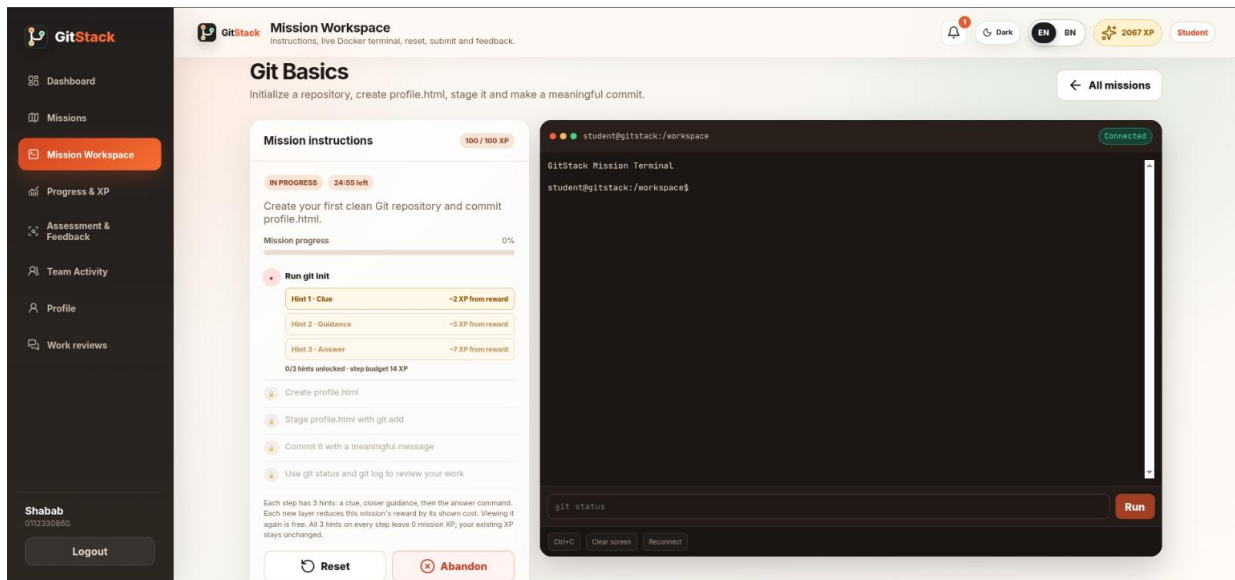


Figure 12. Mission workspace showing step order and the three hint layers with costs.

Technical Notes and Limitations

Hints unlock in order and are state-aware. A request is not charged when the service cannot verify a reliable answer for the active state. Reopening a purchased layer is free. A mission can still be completed after all its hint budget is used, with zero reward remaining for that attempt.

3.3.1 Dynamic XP Allocation

The hint-XP mechanism makes the reward trade-off explicit and keeps it bounded by the mission reward. For a mission reward R and N steps, step i (starting at zero) receives weight $w_i = N + i$. The server distributes R across the steps in proportion to those weights using whole-number allocation, so all step budgets sum exactly to R . Each step budget is divided among Clue, Guidance, and Answer using

weights 1:2:3. Later steps therefore have larger budgets, while more specific help uses a larger share of the current step budget.

Table 4. Example allocation for a 100-XP mission with five steps.

Step	Step budget	Clue cost	Guidance cost	Answer cost
1	14	2	5	7
2	17	3	6	8
3	20	3	7	10
4	23	4	8	11
5	26	4	9	13
Total	100	16	35	49

Why this matters: a fixed charge such as 10 XP for every hint can overwhelm a short mission or unfairly erase XP earned in earlier work. GitStack instead charges only from the reward available in the current attempt, scales help by its specificity and step budget, and preserves the exact mission total. If all three layers are used on every step, the mission can pass with zero reward remaining; existing account XP stays unchanged. [6]

3.4 Docker Sandbox and Live Terminal

Feature Overview

Each individual mission uses a temporary workspace so students can practise Git without changing the host project or another student's files. Team missions use a separate collaboration mode that can reach the assigned Gitea service.

User Flow and Workflow

The server creates or resumes an owned sandbox, starts a shell process, and attaches the browser terminal over an authenticated WebSocket. Students enter commands; the gateway streams output and status back to the page. Reset and stop controls manage the sandbox lifecycle, while expiration and cleanup rules remove idle or old workspaces.

Technologies Used

Docker Engine, a Debian-based sandbox image with Bash and Git, resource-limited containers, WebSocket, xterm.js, and PostgreSQL sandbox-session records.

Technical Implementation

SandboxSession records map users and MissionRun instances to container identifiers, mode, status, and expiry. Individual containers run as a non-root user and have no network. Collaboration containers use the restricted internal network needed for Gitea. The terminal gateway validates the session, origin, user ownership, and running sandbox before attaching the shell. [1, 2, 7]

Technical Notes and Limitations

A Docker sandbox is temporary execution space, not durable storage. For collaboration, pushed commits can be restored from Gitea after workspace recovery; unpushed files in a removed container are not a backup.

3.5 Gitea Team Collaboration

Feature Overview

GitStack supports a three-person team workflow using a real Gitea repository. Each student has a role, a linked Gitea identity, a branch, and a separate workspace. The instructor can prepare a private organization-owned repository and synchronize the eligible members.

User Flow and Workflow

The instructor creates a team and an active Collaboration Basics assignment. GitStack provisions the repository, a Gitea team with write access, role branches, a mission issue, starter files, webhook settings, and per-member runs. Students use their own Gitea accounts to push work and open pull requests. The instructor reviews the event timeline and team report.

Technologies Used

Gitea REST API, organization/team/repository services, signed webhooks, GitEvent persistence, Docker collaboration network, and team/assignment models.

Technical Implementation

The three roles are Feature Developer, Test Developer, and Code Reviewer. The workflow records issues, branches, commits, pushes, pull requests, reviews, requested changes, test evidence, conflict resolution, and ordered merges. Gitea sends signed events; GitStack verifies the HMAC signature, stores normalized event evidence, and deduplicates deliveries. Assessment combines these events with repository-state checks. The rubric separates 70 role-specific points from 30 shared workflow points; passing requires a total of at least 70 and completion of required shared rules. [3, 5]

Feature Screenshot and Output

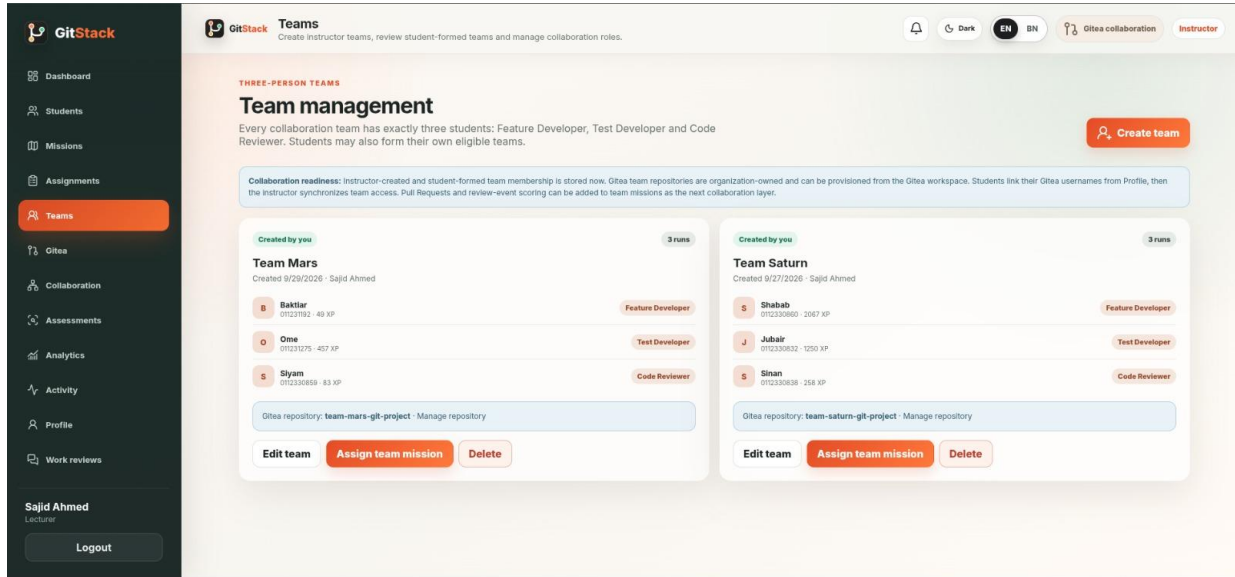


Figure 13. Instructor team management showing the three required collaboration roles.

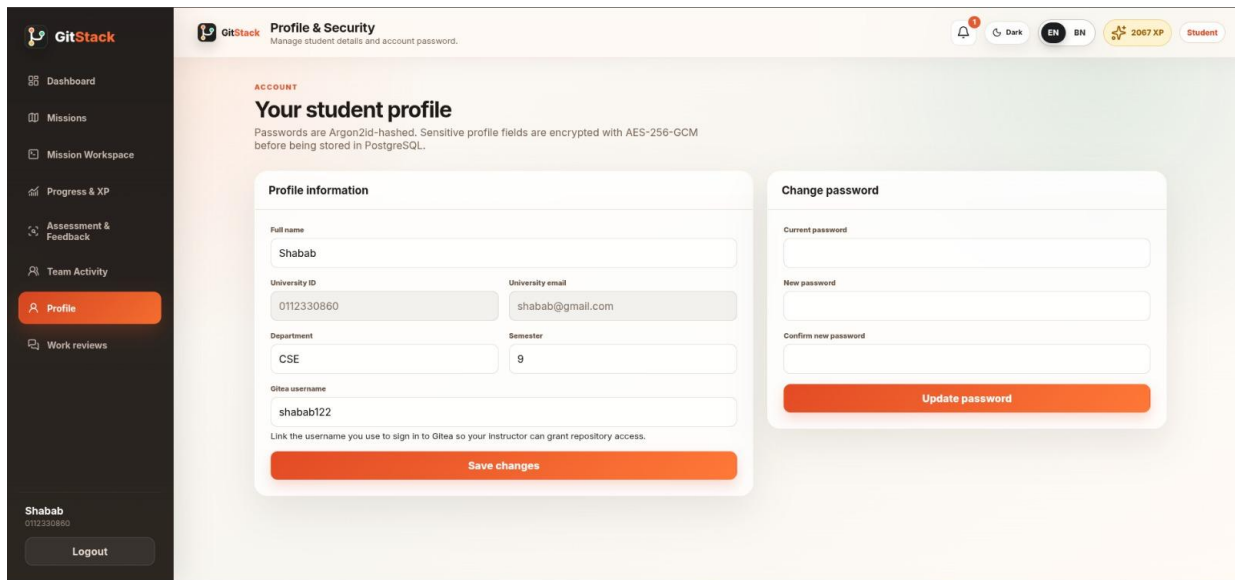


Figure 14. Create Gitea repository dialog with the private-repository option selected.

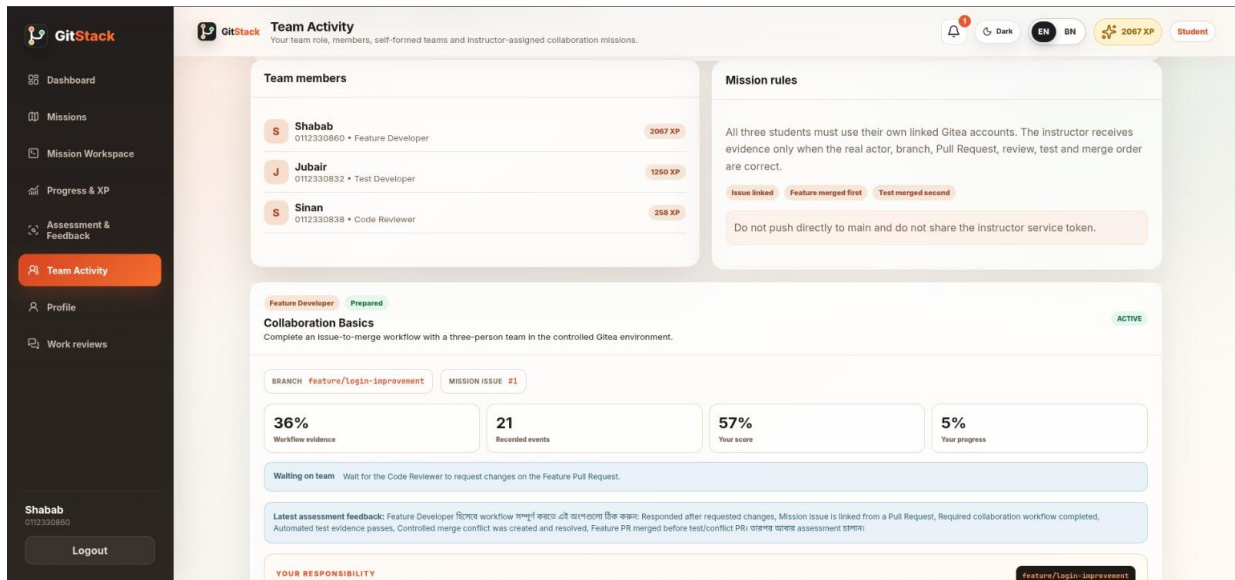


Figure 15. Student Team Activity view with role, assignment state, and workflow progress. Values are sample records.

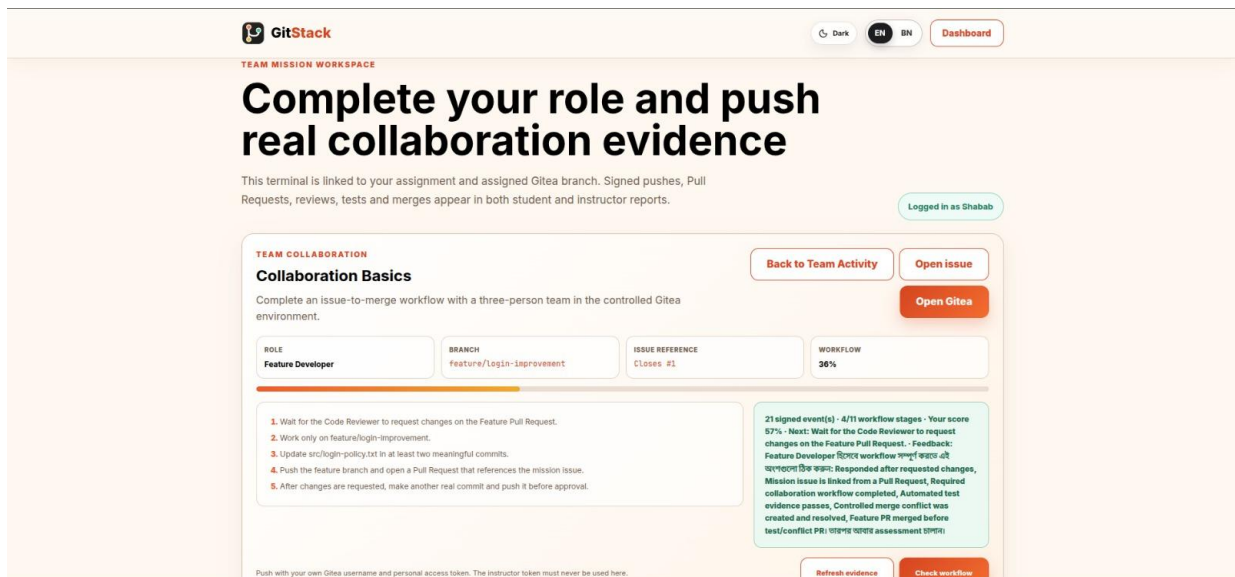


Figure 16. Role-specific team workspace instructions and evidence progress.

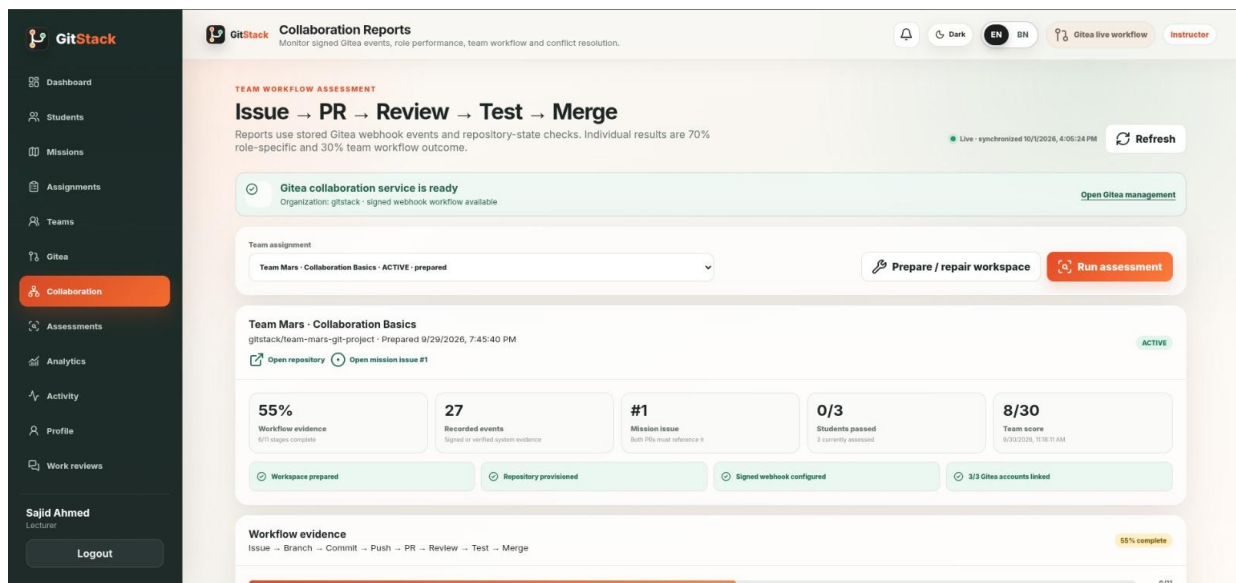


Figure 17. Instructor collaboration report combining team progress and stored Gitea evidence. Values are sample records.

Technical Notes and Limitations

Repository access is synchronized only for students whose Gitea usernames are linked. Students push with personal Gitea credentials; the backend service token is not exposed in the browser terminal.

3.6 Assessment, Feedback, Analytics, and Work Reviews

Feature Overview

GitStack provides individual mission results and team workflow reports so students can see what passed and instructors can see where work needs attention. The platform also supports private work-review requests for a student contribution.

User Flow and Workflow

After a student submits a mission, the server evaluates required repository rules and stores the score and feedback. For team work, it evaluates the role rules and shared workflow evidence. Instructors review result rows, analytics, activity, and collaboration reports. A student can submit a branch, commit, pull request, and selected files for a private instructor review.

Technologies Used

AssessmentResult, Feedback, GitEvent, WorkReviewRequest, and DashboardNotification models; instructor and student REST APIs; Gitea repository evidence; HTML/CSS/JavaScript dashboards.

Technical Implementation

Individual assessment reads observable Git state such as required files, staging, commits, branches, and clean status. Team assessment reconciles stored Gitea events with repository state and merge order. Feedback is persisted in English or Bangla. Review requests pin a submitted commit SHA and evidence so later pushes do not silently replace the reviewed snapshot. The dashboards summarize progress and expose detailed checks. [1, 2, 3]

Feature Screenshot and Output

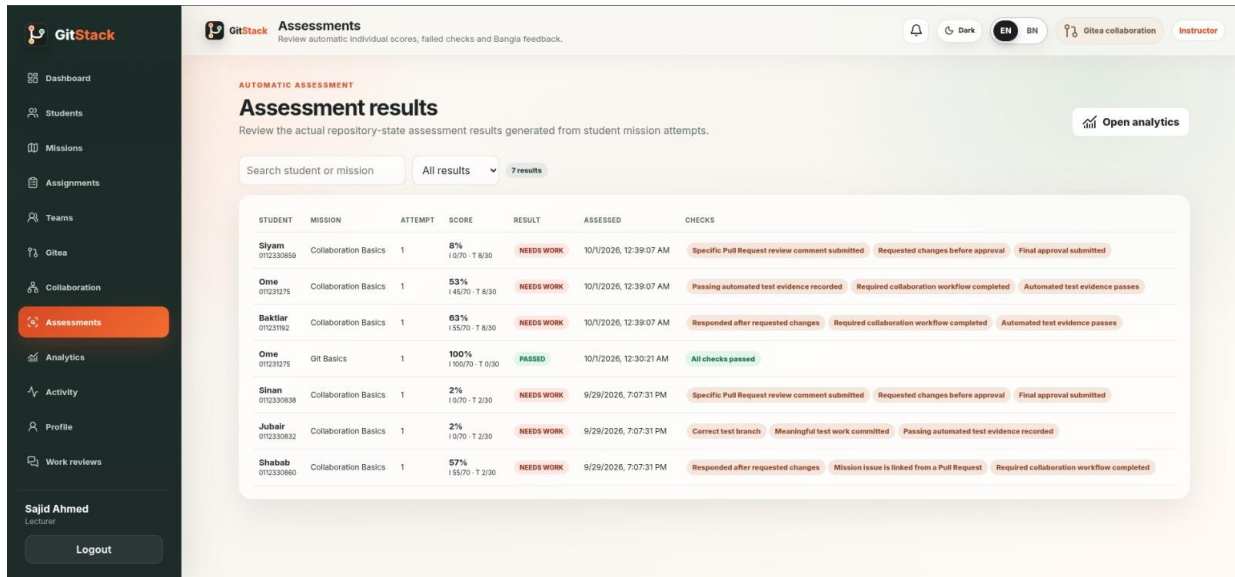


Figure 18. Instructor assessment results with outcome and rule checks. Sample student records are shown.

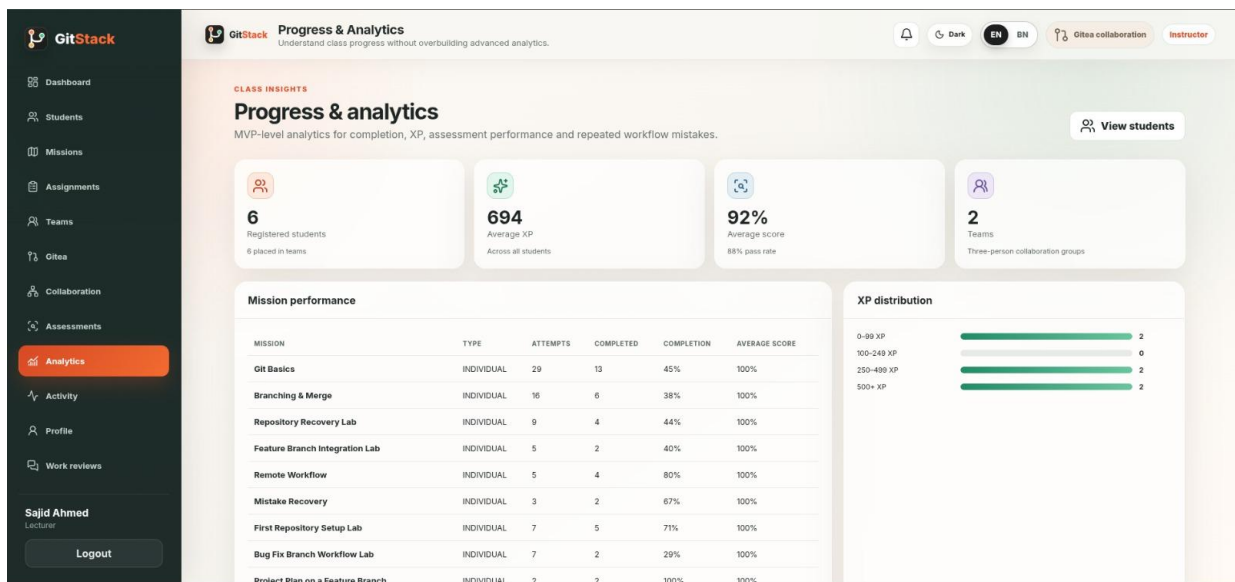


Figure 19. Progress and analytics summary for completion, XP, assessment, and mission performance. Sample records are shown.

GitStack Work Reviews
Individual contributions and instructor feedback.

Work reviews
Request private feedback on your own team contribution and track your instructor's replies.

Request a review
Describe your own portion of the work. Your team instructor receives a private request. The submitted commit snapshot is preserved, so later pushes do not replace it.

Team repository
Team Saturn - gitstack/team-saturn-git-project - Sajid Ahmed

Assignment (optional)
General team work

Review title
Review my login validation

Work reference
Branch

Branch name, commit SHA or PR number
feature/login-improvement

My contribution and questions
Explain what you changed, how you tested it and what you want feedback on.

Files to focus on (optional)
src/login.js
tests/login.test.js

One exact repository-relative path per line, up to 20 files. Leave blank to review the whole submitted change.

Send review request

Figure 20. Student form for requesting private instructor feedback on a team contribution.

Technical Notes and Limitations

Dashboard totals in the supplied still images are demonstration records. They illustrate interface behavior and should not be interpreted as a controlled class evaluation or measured learning outcome.

3.7 Git Flow Lab and Git Hangman

Feature Overview

Git Flow Lab and Git Hangman provide short practice away from graded missions. The first makes Git state changes visible; the second reinforces Git vocabulary through a guessing game.

User Flow and Workflow

In Git Flow Lab, a learner selects or replays a guided command and compares the repository before and after it. In Git Hangman, a learner selects a difficulty, reads a clue, guesses a Git term, and reviews its explanation after the round.

Technologies Used

Browser-side JavaScript, CSS, responsive HTML, a deterministic Git Flow Lab state model, and a Git vocabulary game engine.

Technical Implementation

Git Flow Lab visualizes ten commands: init, clone, status, add, commit, push, pull, branch, checkout, and merge. Its displayed repository state is a simulation; it does not execute in the mission sandbox. Git Hangman contains 24 Git terms and three difficulty levels. Neither tool changes mission progress or awards mission XP. [1, 6]

Feature Screenshot and Output

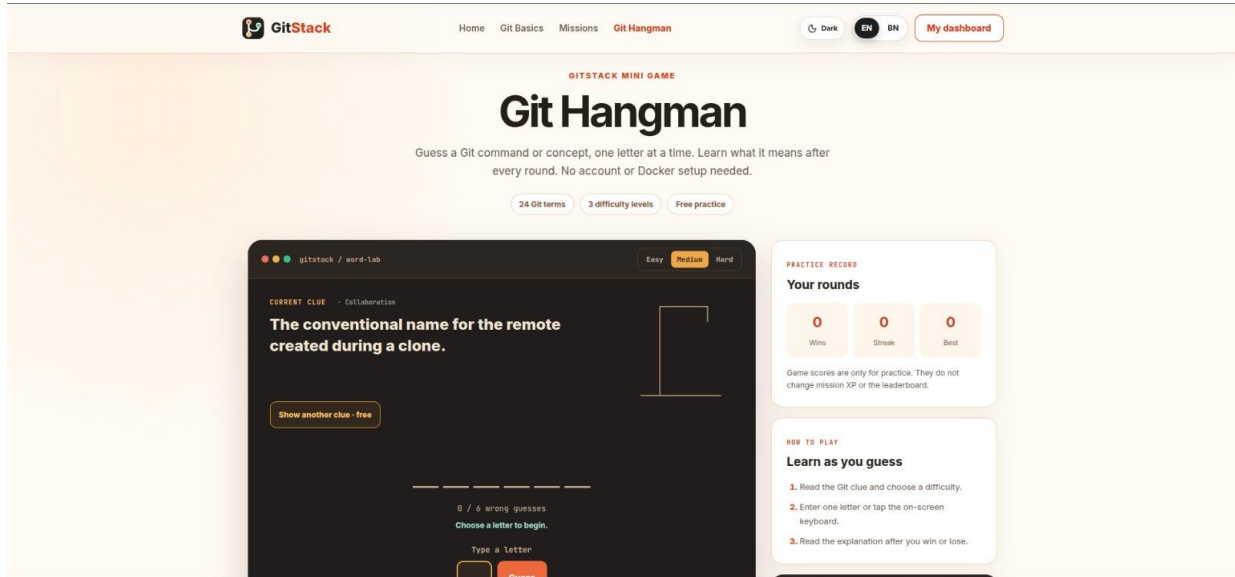


Figure 21. Git Hangman initial game screen with a vocabulary clue.

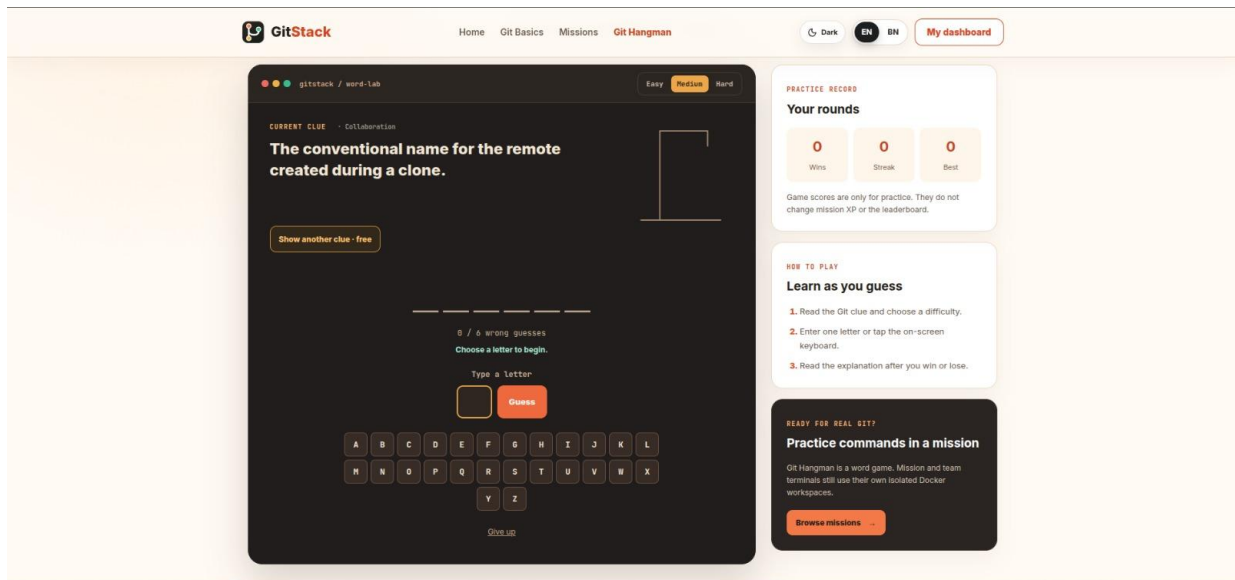


Figure 22. Git Hangman round after a learner begins guessing.

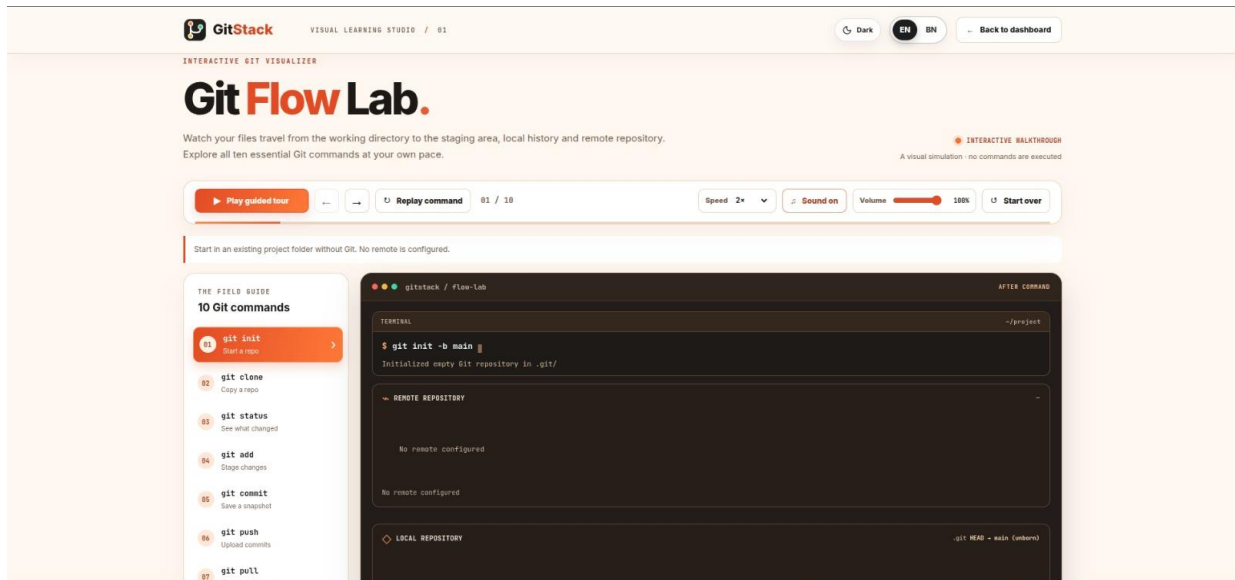


Figure 23. Git Flow Lab with a guided command and before/after repository visualization.

Technical Notes and Limitations

These are supplementary learning tools. The real terminal and Gitea workflow remain the evidence sources for mission assessment.

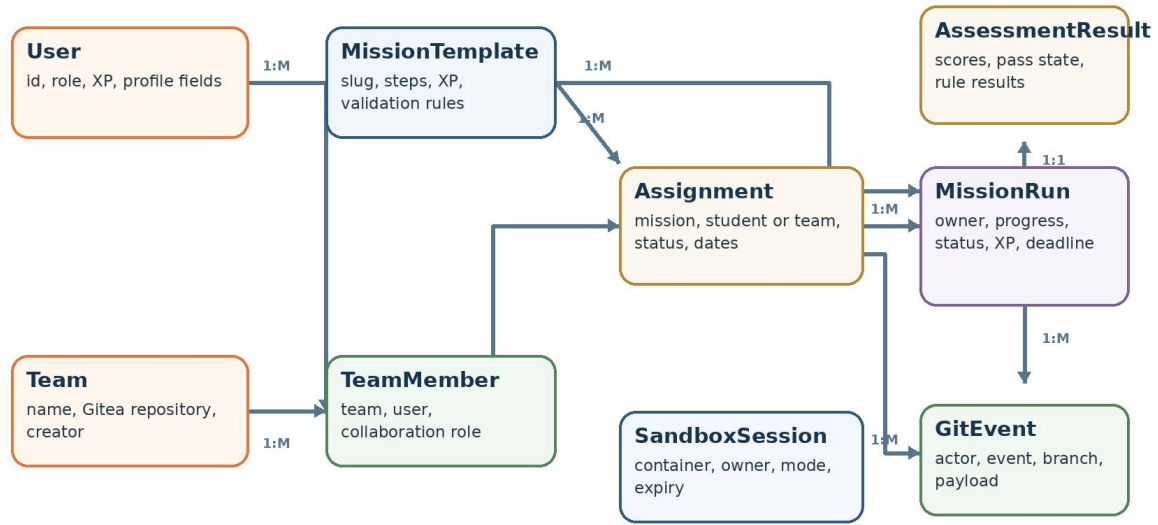
4. Database and Data Management

4.1 Database Technology

GitStack uses PostgreSQL for persistent application data and Prisma as its ORM and migration system. Gitea has a separate PostgreSQL service and its own repository data volume. This separation keeps GitStack learning records distinct from Git hosting state. [1, 2]

4.2 Database Schema and ER Diagram

Core GitStack Data Relationships



Core relationships shown. MissionHintUse, Feedback, WorkReviewRequest and DashboardNotification extend this model.

Figure 24. Core user, team, mission, assignment, run, sandbox, event, and assessment relationships.

The schema uses UUID identifiers, foreign keys, unique constraints, and indexes for common ownership and status lookups. The GitEvent Gitea event identifier is unique so duplicate webhook delivery cannot create duplicate evidence. Hint usage is unique per mission run and step. The diagram shows the main relationships; review, hint, feedback, and notification entities extend the model. [2]

4.3 Key Tables and Models

Table 5. Principal Prisma models.

Model	Purpose	Representative data
User	Account, role, profile, and XP.	role, password hash, encrypted profile fields, giteaUsername, xp
Team / TeamMember	Three-person team and assigned roles.	team name, Gitea repository, user ID, collaboration role
MissionTemplate	Built-in or instructor-created mission.	slug, steps, XP reward, validation rules, publish state
Assignment	Mission assigned to a student or team.	target, creator, status, start/due dates, collaboration state
MissionRun	One attempt or team role run.	status, progress, attempt, expiry, XP, repository link
MissionHintUse	Hint layer and XP cost for a step.	run ID, step index, layer, cost
SandboxSession	Temporary container lifecycle.	container ID, owner, mode, status, expiry
GitEvent	Verified Gitea workflow event.	actor, event type, branch, delivery ID, payload

Model	Purpose	Representative data
AssessmentResult / Feedback	Scores, rule results, and corrective guidance.	role/team scores, pass state, message, language
WorkReviewRequest / Notification	Private review history and dashboard events.	commit SHA, evidence, reviewer feedback, read state

4.4 Data Flow

Registration creates a User record with the selected role. Mission start creates or resumes a MissionRun and SandboxSession. Hint use is recorded against the active run and step. Submission creates an AssessmentResult and Feedback record and updates XP only when the one-time reward rules permit it. In a team assignment, repository events arrive through the signed webhook route and are stored as GitEvent rows; the assessment service combines those rows with Gitea repository state. Gitea stores its own accounts and repositories outside the GitStack application database.

5. API and Backend Design

5.1 API Overview

The backend is an Express application organized around authentication, student, instructor, sandbox, Gitea, work-review, and notification route families. Authenticated routes derive the actor from the session and enforce the required role and ownership. The public Gitea webhook endpoint uses signature verification instead of a user session. The browser terminal uses a separate WebSocket gateway for interactive shell I/O. [3]

5.2 Key Endpoints

Table 6. Representative API operations from the current route documentation.

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create a role-based account.	Public, rate limited
POST	/api/auth/login	Create an authenticated session.	Public, rate limited
GET	/api/missions	List published mission templates.	Public catalogue
GET	/api/student/dashboard	Load a student summary.	Student
POST	/api/student/missions/:slug/start	Start or resume an individual mission.	Student, owned run
POST	/api/student/mission-runs/:id/hints/:stepIndex	Request the next valid hint layer.	Student, owned run
POST	/api/student/mission-runs/:id/submit	Submit repository state for assessment.	Student, owned run
POST	/api/instructor/missions	Create an instructor-authored mission.	Instructor
POST	/api/instructor/assignments	Assign a published mission to a student or team.	Instructor

Method	Endpoint	Purpose	Access
POST	/api/instructor/assignments/:id/prepare-collaboration	Prepare or repair team collaboration.	Instructor
POST	/api/gitea/repositories	Create an organization-owned team repository.	Instructor
POST	/api/gitea/webhook	Receive signed Gitea events.	HMAC signature

5.3 Authentication and Authorization

Registration and login issue a signed JWT in an HttpOnly cookie. Passwords are hashed with Argon2id. Student and instructor APIs require an authenticated session; route handlers check role, record ownership, or team membership before returning protected information or accepting changes. The Gitea service token is used by backend integrations and must remain in private environment configuration. Webhook requests are accepted only after validating the HMAC signature over the request body. [1, 3]

6. Frontend and Client-Side Implementation

The current repository implements the browser experience with separate HTML pages, shared CSS, and vanilla JavaScript modules. It does not use the Next.js/Tailwind stack from early planning documents; this report follows the code in the supplied latest archive. Student and instructor pages include dashboards, mission catalogues, profiles, assessment, teams, collaboration, and analytics. Shared scripts manage navigation, theme state, English/Bangla labels, notifications, and common authenticated requests. [1]

Mission pages render ordered instructions and progress beside an xterm.js terminal. The terminal client connects to the authenticated WebSocket, streams user input, and displays output and connection status. Forms call REST endpoints and render server-side validation or error responses. The client presents results, but the server remains responsible for step validation, XP accounting, repository checks, and access control.

Table 7. Main browser areas.

Client area	Implemented responsibility
Public learning pages	Landing page, Git lessons, login, and registration.
Student workspace	Dashboard, mission catalogue and terminal, progress, assessment, team activity, profile, and reviews.
Instructor workspace	Dashboard, students, missions, assignments, teams, Gitea, collaboration reports, assessments, analytics, activity, and profile.
Supplementary learning	Git Flow Lab, Git Hangman, theme and language controls.

7. Security and Error Handling

GitStack applies security controls at the account, API, webhook, and sandbox boundaries. These controls are appropriate for a controlled laboratory environment, but they do not make the current local configuration a production deployment.

Table 8. Security controls and deployment notes.

Area	Implementation and operational note
Passwords and sessions	Argon2id password hashing; signed JWT session in an HttpOnly cookie; role checks on protected routes.
Input and ownership	Zod validates request payloads; student records, mission attempts, and team access are checked on the server.
Profile data	Selected profile fields use AES-256-GCM encryption and HMAC lookup hashes. Keep DATA_ENCRYPTION_KEY private and consistent with its database.
Gitea integration	Backend service token is kept in private environment configuration. Webhooks require HMAC signature verification and duplicate events are deduplicated.
Sandbox isolation	Non-root student container, CPU/memory/PID limits, workspace/temp limits, TTL cleanup, and separate individual/collaboration networks.
Local deployment boundary	Local HTTP development is not TLS-protected. Optional read-only-root and capability-hardening flags are disabled by compatibility defaults and need host testing before enabling.

When a mission command or submission is rejected, the server returns a structured result so the client can show the failed step or rule and let the student correct the repository. Invalid roles and ownership do not grant access to another student's attempt. Configuration values and access tokens are intentionally omitted from this report.

8. Testing and Validation

8.1 Testing Approach

The archive includes syntax checks, UI and feature structure checks, focused student and instructor checks, mission runtime tests, Docker sandbox tests, Gitea checks, and regression scripts for hints, XP, collaboration, terminal handling, and assessment. The fresh setup log from Jubair's Ubuntu machine provides the observed local results below. A setup check is not treated as a substitute for future usability, accessibility, or learning-outcome evaluation.

8.2 Test Cases and Observed Results

Table 9. Local validation evidence recorded during fresh setup on 4 October 2026.

Test case	Input or scenario	Expected result	Observed result	Status
Source syntax	Setup source check	All JavaScript files parse.	Syntax check passed for 140 JavaScript files.	Pass
UI structure	Dashboard and HTML checks	Expected pages and controls exist.	21 dashboard pages and 32 HTML pages passed UI checks.	Pass
Feature structure	Feature update checks	Feature markup is consistent.	Checks passed across 32 HTML pages.	Pass

Test case	Input or scenario	Expected result	Observed result	Status
Student and instructor	Role workflow checks	Dashboards, missions, assignments, and teams behave as expected.	Student mission/encryption helpers and instructor assignment/team checks passed.	Pass
Sandbox	Doctor, image build, lifecycle smoke test	Docker image and workspace commands work.	Sandbox doctor ready; image built; lifecycle and command smoke test passed.	Pass
Database	Prisma validate and deploy	Schema is valid and migrations apply.	Schema valid; 13 migrations applied; five mission templates seeded.	Pass
Gitea integration	npm run gitea:doctor	Authentication and required repository permissions work.	Authentication, organization create, repository write, and cleanup checks passed.	Pass
Application startup	npm run dev and browser check	GitStack home page loads on localhost.	The browser displayed localhost:3000/home.html.	Pass

8.3 Evidence and Evaluation Boundary

The supplied project still images demonstrate the implemented screens and user flows. Dashboard figures in those captures contain sample records and are not test measurements. The local setup transcript provides the environment test results. The team should still rehearse the final demonstration on the presentation laptop, including an individual mission completion and a team repository preparation, because these depend on live Docker, PostgreSQL, Gitea, and account configuration.

9. Deployment and Setup

9.1 Deployment Architecture

The verified target is a local Ubuntu/Linux host with Node.js, npm, Docker Engine, and Docker Compose. GitStack runs as a Node/Express process on port 3000. Compose runs GitStack PostgreSQL, Gitea PostgreSQL, and Gitea; Gitea exposes HTTP on port 3002 and SSH on port 2222. Docker creates separate sandbox containers as students start workspaces. The application database, Gitea database, and Gitea repository data use separate named volumes. [1]

Table 10. Local service endpoints.

Service	Default address	Purpose
GitStack	http://localhost:3000	Student and instructor application.
Gitea HTTP	http://localhost:3002	Repository and collaboration web interface/API.
Gitea SSH	localhost:2222	Git SSH transport for configured student accounts.
GitStack PostgreSQL	localhost:5432	Application accounts, missions, assignments, and results.

9.2 Fresh Local Setup

For a fresh machine, extract the archive, install the locked dependencies, run the setup script, complete Gitea's initial installation, configure the private integration token, and start the application. The current installation followed this sequence:

```
npm ci
npm run setup -- --rebuild
npm run gitea:doctor
npm run dev
```

Before the first GitStack startup, an administrator completes Gitea setup at <http://localhost:3002> and puts the integration token and owner in the local `.env` file. The doctor script verifies that the API token can authenticate, create the organization, and write and clean up a test repository. Secret values must not be added to the repository or report.

9.3 Environment Configuration

Table 11. Environment variable groups. Values and secrets are intentionally not reproduced.

Variable group	Configuration purpose
APP_ORIGIN, PORT, NODE_ENV	Application origin, listener port, and runtime mode.
DATABASE_URL	GitStack PostgreSQL connection.
JWT_SECRET, JWT_EXPIRES_IN, AUTH_COOKIE_NAME	Session signing and cookie settings.
DATA_ENCRYPTION_KEY	Encryption key for protected profile fields; retain with the matching database.
GITEA_BASE_URL, GITEA_INTERNAL_BASE_URL, GITEA_ADMIN_TOKEN	Gitea external/internal API endpoints and private service credential.
GITEA_ORGANIZATION, GITEA_WEBHOOK_SECRET, GITEA_WEBHOOK_TARGET_URL	Organization and signed event delivery configuration.
SANDBOX_* and DOCKER_SOCKET_PATH	Sandbox image, resource limits, lifecycle, networking, and Docker connection.

10. Team Contribution

The following allocation reflects the project team's stated areas of responsibility. It describes the intended module ownership and does not assign commit counts or claim that a module was written by only one person.

Table 12. Team member responsibilities.

Team member	Role / responsibility	Major contribution area
Jubair Ahmed Efty	Backend, Gitea, and Docker	Backend APIs and mission services; Gitea repository and collaboration integration; Docker sandbox and terminal infrastructure.
Shabab Ahmed	Frontend and backend	Student and instructor interface; mission and dashboard flows; integration between browser workflows and backend services.

Team member	Role / responsibility	Major contribution area
Siam Ibne Sarwar	Database	PostgreSQL and Prisma data model; persistence relationships, schema updates, migrations, and database support.

11. Challenges and Technical Decisions

11.1 Major Challenges

The system must verify Git work that can be completed with more than one valid command, while preventing students from skipping required steps. It must also give useful hints without charging for an invalid answer, connect local student sandboxes to a real Git hosting service, and report workflow evidence consistently across three separate roles.

11.2 Solutions and Design Decisions

Table 13. Key implementation decisions and their effect.

Challenge	Decision and effect
Exact command checking can reject valid work.	Validate repository state and use an ordered step gate; allow equivalent Git commands when they reach the required state.
Repeated hints can make rewards feel arbitrary.	Allocate XP deterministically across steps and hint layers; charge from the current mission attempt only.
A fake collaboration screen cannot prove Git teamwork.	Use Gitea repositories, branches, issues, pull requests, reviews, tests, and signed webhook evidence.
A shared shell risks cross-student changes.	Give each student an owned non-root Docker workspace and isolate individual mission networking.
Dashboard claims need traceable evidence.	Persist assessment checks and feedback; keep sample dashboard figures separate from test results.

11.3 Known Limitations

- The current system is locally hosted and uses HTTP for development; a public deployment needs TLS, secure secrets management, tested backups, and stricter registration policy.
- The evaluator checks defined repository and workflow rules; it is not a general semantic code-quality or continuous-integration service.
- Docker hardening options are available but disabled in compatibility defaults and need a tested rollout.
- The npm install summary reported 11 advisories (1 moderate and 10 high); dependency-level triage and remediation remain necessary before a public release.
- The project has not yet reported a controlled accessibility review or measured learning outcomes.

12. Conclusion and Future Improvements

12.1 Conclusion

GitStack combines individual Git practice, instructor-authored missions, Docker-backed terminals, state-aware hints and XP, repository-state assessment, and real Gitea collaboration in one laboratory platform. Students receive a safe place to practise and inspect their work; instructors can create assignments and review recorded evidence. The latest local setup completed its source, UI, sandbox, migration, and Gitea checks on Ubuntu. The separation between ungraded learning tools, isolated individual missions, and real team repositories gives each exercise a clear purpose.

12.2 Future Improvements

- Review npm audit findings and update affected packages with regression testing.
- Plan and rehearse production hardening, TLS, instructor approval, backup and restore, and sandbox capability restrictions.
- Add a documented browser-based acceptance checklist for all critical student and instructor journeys.
- Run an accessibility review of keyboard use, contrast, screen-reader labels, and bilingual content.
- Evaluate learning outcomes with a defined student cohort and report the method and results separately from sample dashboard data.

13. Project Links and References

13.1 Project Links

GitHub repository: <https://github.com/Shabab122/GitStack>

Local demonstration: <http://localhost:3000>. The current build has no public live-application URL.

Gitea demonstration service: <http://localhost:3002> on the configured local host.

13.2 References

1. [1] GitStack project repository, README.md, package.json, docker-compose.yml, and RUN_COMMANDS.md, archive dated 4 October 2026.
2. [2] GitStack project repository, prisma/schema.prisma and prisma/migrations/.
3. [3] GitStack project repository, docs/api-documentation.md, routes/, and services/gitea/.
4. [4] GitStack project repository, docs/mission-design.md, docs/PUBLISHED_MISSION_RUNTIME_V33.md, and instructor mission routes.
5. [5] GitStack project repository, docs/GITEA_COLLABORATION_V18.md, docs/MISSION_ASSIGNMENT_COMPLETE.md, and services/collaboration/.
6. [6] GitStack project repository, docs/THREE_LAYER_HINTS_AND_FLOW_LAB.md, docs/SYSTEM_COMMAND_HINTS_V35.md, and services/student/mission-hint-xp.js.
7. [7] GitStack project repository, docs/SANDBOX_SERVICE.md and services/sandbox/.
8. [8] Local Ubuntu setup transcript supplied by Jubair Ahmed Efty: fresh setup, Prisma migrations, Gitea doctor, and application smoke check, 4 October 2026.